

The AI Cell Model: Decentralized Compute and Modular Fault Containment as an Alternative to Monolithic Shared-Parameter Architectures

Daysun Simeon — Celos Labs

Working paper — v1.0 (software-only evidence). Not yet peer reviewed.

September 2026

Abstract

Modern AI systems are built two ways at two different layers, and both share the same failure mode: a single shared resource serves many independent tasks. At the hardware layer, GPU/von Neumann accelerators separate compute from memory and move data over a shared bus. At the model layer, many deployed systems serve many users or many tasks from one shared set of parameters. Both designs are efficient when nothing goes wrong. Both concentrate risk: a bus saturated by concurrent traffic, or a parameter update that corrupts one shared model, degrades every task the shared resource serves.

We propose the **AI Cell** — a unit defined as the co-location of compute, memory, state, and routing — as a primitive for building systems that trade some of that shared-resource efficiency for graceful degradation under load and under failure. We do not claim

this trade is free, or that it wins in every case. We report two small, real, reproducible experiments that test where it does and does not win: (1) a routing/concurrency simulation comparing a decentralized mesh against a shared-bus baseline, and (2) a fault-injection experiment comparing independently-parameterized agents against a shared-parameter baseline. Both are software-only; no custom hardware exists yet. We report results honestly, including the case where the AI Cell model performs worse, and outline what would be needed to test the claim at production scale.

1. The Problem

Two long-standing observations motivate this work:

1. **The von Neumann bottleneck.** Separating compute and memory means every operation pays a data-movement cost. This has been a known architectural tax since Backus's 1977 critique of the von Neumann style, and it reappears today as the HBM/interconnect bandwidth limits that bound GPU-based accelerators under memory-bound workloads.
2. **The blast radius of shared parameters.** A model or parameter set serving many downstream tasks or users concentrates the impact of any single fault — a bad update, a corrupted checkpoint, a poisoned gradient — across every task it serves. Reliability engineering has a name for this in distributed systems generally: shared state is a single point of correlated failure. It's less commonly discussed at the model-parameter level.

Both problems have the same shape: sharing a resource is efficient in the average case and fragile in the tail case (high concurrency, or the moment something breaks).

2. The AI Cell Model

We define an **AI Cell** as a unit that co-locates four properties:

AI Cell = Compute + Memory + State + Routing

- **Compute** — the cell's own processing, not dispatched to a shared processor.
- **Memory** — local, private state storage, not a shared/global address space.
- **State** — the cell's current condition, readable by neighbors but owned only by the cell itself.
- **Routing** — the ability to pass events to adjacent cells directly (peer-to-peer), without a central dispatcher.

A **fabric** is a collection of AI Cells connected in a topology (in our current implementation, a 2D grid with 4-neighbor connectivity, addressed by explicit (x, y) coordinates). An event entering the fabric travels hop-by-hop between neighboring cells rather than through a shared bus or central coordinator. No individual cell has global knowledge of the fabric; each only knows its own neighbors.

This is deliberately a modest, mechanical definition. We are not claiming a new theory of computation — the underlying ideas (message-passing between independent local units, no shared global state) are decades old (see Section 3). The contribution is applying this pattern deliberately across both the model-parameter layer and the execution/routing layer, and testing where the resulting trade-offs actually land.

3. Related Work

The AI Cell model does not invent new primitives; it recombines several well-established ones:

- **The Actor Model** (Hewitt, Bishop, and Steiger, 1973) — computation as independent actors communicating only via

asynchronous messages, with no shared mutable state. AI Cells are actors specialized for local compute + memory + routing.

- **Cellular automata** (von Neumann's self-reproducing automata; Conway's *Game of Life*) — a grid of cells whose next state depends only on local neighbors. Our fabric's coordinate-addressed neighbor topology is structurally a cellular automaton substrate carrying event-driven messages instead of a fixed update rule.
- **Neuromorphic computing** (Mead, 1990, and the subsequent line of spiking neural network hardware) — co-locating compute and memory at the neuron level specifically to avoid von Neumann data movement costs. The AI Cell model applies the same motivation in software, ahead of any custom silicon.
- **Multi-agent reinforcement learning** (see Buşoniu, Babuška, and De Schutter's 2008 survey for the foundational taxonomy) — independent versus shared/centralized policies among cooperating agents is a studied trade-off; we borrow this framing directly for our fault-containment experiment in Section 4.2.

What we believe is under-explored is treating "shared vs. independent parameterization" and "shared bus vs. decentralized routing" as the *same* architectural decision applied at two different layers of an AI system, and measuring the failure/concurrency trade-off explicitly rather than only the average-case efficiency trade-off that usually motivates parameter sharing.

4. Evidence

Both experiments below are implemented in Python (`cellos-emulator` v0.1) and are fully reproducible; code and raw results are included with this paper. Neither experiment involves custom hardware. Absolute timing numbers reflect commodity hardware running Python and should not be read as hardware latency claims

— the meaningful signal in each case is the *relative* comparison, run in the same process, on the same machine, under the same conditions.

4.1 Fault Containment: Independent vs. Shared Parameters

Setup. Eight independent task instances (a 10×10 grid-navigation task, solved via tabular Q-learning) were trained under two conditions with an equal total training budget (24,000 episodes either way):

- **Monolithic:** all 8 instances shared one Q-table, updated by pooled experience from all instances.
- **Modular:** each instance had its own independent Q-table, trained only on its own experience.

Both conditions were evaluated under a tight step budget (20 steps, against a worst-case optimal path of 18) so that success requires a genuinely accurate policy rather than a lucky random walk — an earlier, looser version of this experiment (40-step budget on a 5×5 grid) failed to produce a meaningful signal because even a heavily damaged policy could stumble to the goal in time. We report this because it matters: a fault-injection experiment is only informative if the task is hard enough that damage actually shows up.

After both conditions converged to 100% baseline success, we corrupted 50% of the entries of *one* Q-table: the single shared table in the monolithic condition, or a single instance's table in the modular condition.

Results.

Condition	Baseline success	Post-failure success (system-wide)
Monolithic (1 shared table)	100%	61.4%
Modular (8 independent tables)	100%	93.6%

In the modular condition, the corrupted instance dropped to 49% while the other seven remained at 100% — the damage stayed contained to the unit it hit. In the monolithic condition, the same magnitude of corruption degraded all eight instances simultaneously, because all eight read from the same table.

Honest caveat. This is a small, tabular, single-task toy problem — not a demonstration on a production-scale model. We also note the training process itself is *not* symmetric: the shared table benefited from pooled experience across all 8 instances during training, which is a real advantage of parameter sharing that this experiment does not erase. The claim this experiment supports is narrow and specific: *given equivalent baseline performance, a single fault of fixed severity does less system-wide damage under independent parameterization than under shared parameterization*. It does not show that modular systems train faster, use less data, or perform better in the absence of failure — the opposite is plausible, and worth testing directly in future work.

4.2 Concurrency and Routing: Mesh vs. Shared Bus

Setup. We implemented a `Fabric` (decentralized mesh, 4-neighbor grid, breadth-first hop routing between arbitrary source/target cells) and a `CentralBus` baseline (every event routed source → hub → target, a stand-in for a shared/monolithic dispatcher), over identical grid layouts, in the same process.

Single-event latency (200 trials per grid size, wall-clock):

Grid	Fabric mean latency	Bus mean latency	Fabric mean hops
4×4	4.7 μ s	0.6 μ s	2.6
8×8	14.6 μ s	0.8 μ s	4.9
16×16	61.6 μ s	1.1 μ s	9.9

For a single, isolated event, the bus is faster in every case we tested — a fixed 2-hop dispatch beats a variable-length mesh path, and our reference implementation recomputes the shortest path on every call rather than caching it. **We are reporting this even though it does not favor our own architecture**, because it does.

What the mesh does deliver, at every grid size, is a dramatically smaller *active-cell fraction* — only 4.2% of cells were touched by a routing event on the 16×16 grid, versus 100% for the bus (which, structurally, occupies the whole shared resource for every transfer). This is a real, measured property of the two topologies; we present it as a proxy for the "only active cells draw power" argument for spatial fabrics, not as a wattage measurement, since no physical hardware exists yet to measure that directly.

Concurrent-load completion time (structural discrete-event simulation; see Appendix A for the exact modeling assumptions — this is not a raw wall-clock parallel-hardware measurement):

Concurrent events	Fabric (ticks)	Bus (ticks)	Bus/Fabric ratio
1	10.3	2.0	0.19×
4	17.1	8.0	0.47×
16	21.8	32.0	1.47×

Concurrent events	Fabric (ticks)	Bus (ticks)	Bus/Fabric ratio
64	25.2	128.0	5.09×
128	26.0	256.0	9.84×

A shared bus can only service one hop-step at a time, so its completion time scales linearly with the number of concurrent events. A decentralized mesh only serializes traffic that physically collides on the same cell, so its completion time grows far more slowly. There is a **crossover point around 8–16 simultaneous events** on a 16×16 grid, beyond which the mesh wins, with the advantage widening rapidly.

Honest caveat. This is a structural model of contention, not a literal multi-threaded hardware benchmark — Python's own execution model makes that distinction necessary to state plainly. The assumption being tested (a shared resource serializes; independent resources don't, except where they actually collide) is the real architectural claim, but its precise numeric shape here should not be read as a hardware performance prediction. It does, however, map onto the stated beachhead use case for this architecture — edge vision reflex logic, where many regions of a sensor can plausibly fire near-simultaneously — which is exactly the concurrent regime where this experiment shows an advantage, and exactly the regime a single-event benchmark would miss.

5. Architecture and Roadmap

Consistent with the evidence above, development is sequenced software-first:

1. **Research & whitepaper** (this document).
2. **cellos-emulator v0.1** — the Python reference implementation used for the experiments above (routing,

benchmark harness, fault-injection harness); not yet a released PyPI package, source available alongside this paper.

3. **FPGA emulation & edge-vision reflex benchmarks** — planned; would be the first hardware-adjacent validation of the concurrency claim in Section 4.2.
4. **Custom silicon tape-out** — planned, contingent on prior-phase results.

A production-grade kernel runtime (the roadmap targets Rust) has not been built. The v0.1 emulator described here is a Python reference implementation only, intended to validate the architectural claims cheaply before investing in a lower-level implementation.

6. Limitations

We list these directly rather than leaving them implicit:

- Both experiments are toy-scale (grid sizes up to 16×16; 8 task instances; tabular, not deep, reinforcement learning).
- No real hardware, custom silicon, or even a multi-threaded runtime was used — all reported timings are single-process Python behavior.
- The concurrency benchmark is a structural/discrete-event model, not a measurement of true parallel hardware execution.
- The fault-containment experiment tested one failure mode (random parameter zeroing at one severity level) on one task; we have not tested adversarial failures, gradual drift, or partial/soft corruption.
- No independent replication, peer review, or external audit has occurred.

7. Future Work

- Repeat the fault-containment experiment with function-approximated (deep) policies at larger scale, and with a range of failure severities rather than one fixed corruption fraction.
- Extend the concurrency benchmark to real multi-process or multi-machine execution to validate that the structural contention model holds under true parallelism.
- Build the FPGA emulation phase to obtain the first hardware-adjacent timing and power measurements, replacing the active-cell-fraction proxy with a real measurement.
- Test the AI Cell model directly against the ML-infra reliability use case (model-serving fault isolation) that motivates this research program, rather than the toy navigation task used here as a stand-in.

References

1. Backus, J. (1978). "Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs." *Communications of the ACM*, 21(8).
2. Hewitt, C., Bishop, P., & Steiger, R. (1973). "A Universal Modular ACTOR Formalism for Artificial Intelligence." *IJCAI*.
3. von Neumann, J. (1966). *Theory of Self-Reproducing Automata* (ed. A. Burks).
4. Mead, C. (1990). "Neuromorphic Electronic Systems." *Proceedings of the IEEE*, 78(10).
5. Buşoniu, L., Babuška, R., & De Schutter, B. (2008). "A Comprehensive Survey of Multiagent Reinforcement Learning." *IEEE Transactions on Systems, Man, and Cybernetics*, 38(2).

Appendix A — Reproducibility

All code, exact hyperparameters, random seeds, and raw JSON results for both experiments in Section 4 are provided alongside this paper (`cellos-emulator` v0.1). Key parameters:

- **Fault-containment experiment:** 10×10 grid, 8 instances, 3,000 training episodes/instance, 200 evaluation episodes, 50% corruption fraction, seed 7.
- **Concurrency experiment:** 16×16 grid, concurrency levels {1, 4, 16, 64, 128}, 50 trials per level, seed 42.

Anyone can re-run these experiments with the attached code and should get materially the same qualitative results, given the same seeds.